



Modern Tech in Games

Dylan Fries

Winter 2023



Introduction

SUMMARY

- Collisions are used for detecting when two objects intersect within the game world.
- They are a separate but related system to the Physics system, which handles calculating forces and movement (when it is enabled)
- Collisions come in several forms and can be intimidating to get started with.

FURTHER READING

- I will be pulling most of this content from the Unreal Documentation Here:
- <https://docs.unrealengine.com/4.26/en-US/InteractiveExperiences/Physics/Collision/>

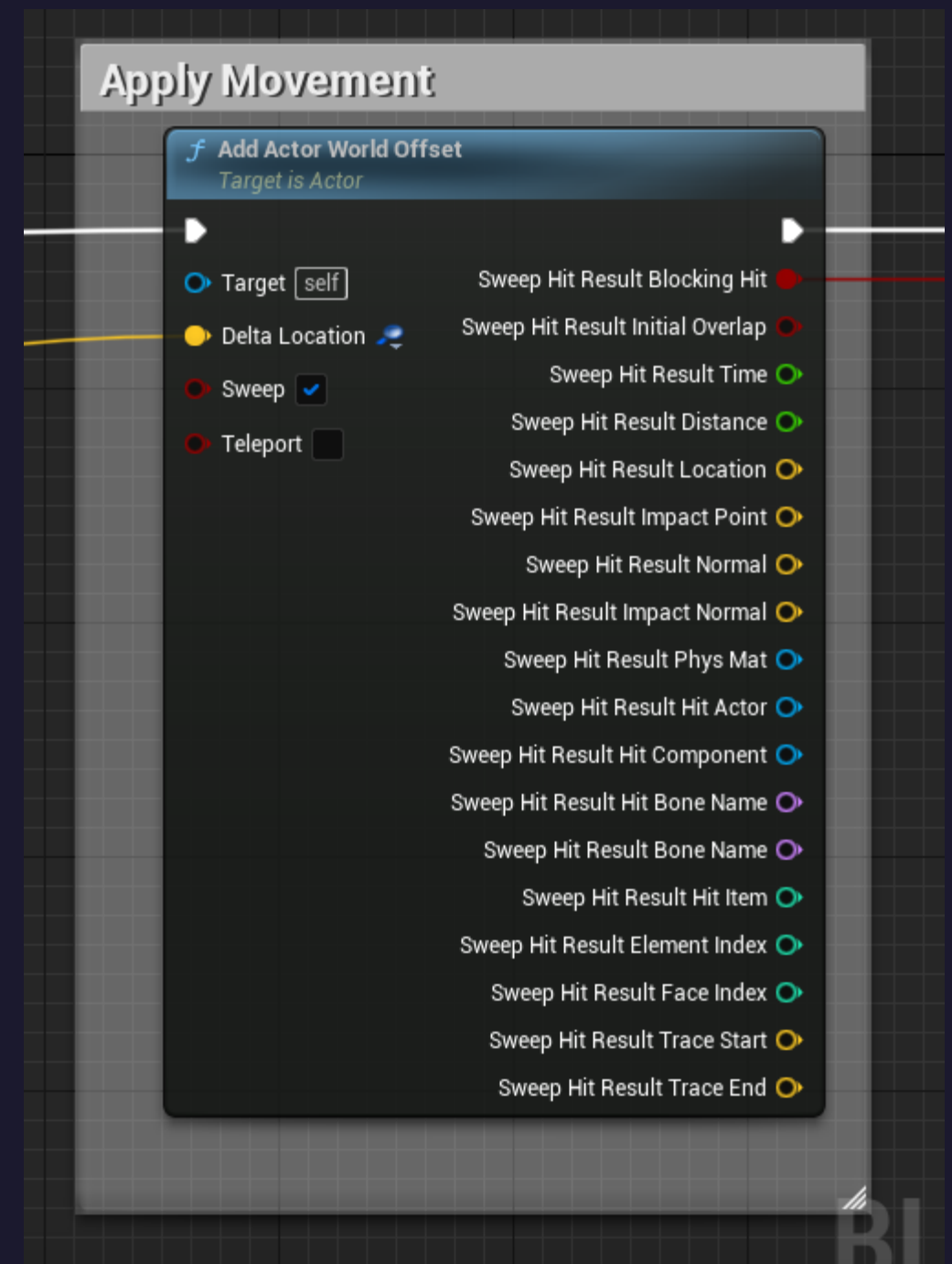
Collision Responses & Trace Responses

- When two collision enabled objects intersect in the world, a collision event is generated.
- This collision event has useful information related to the collision itself (next slide).
- There are two main kind of responses we will encounter when dealing with collision detection
 - Blocking - The objects will stop each other from intersecting. They are treated as a solid.
 - Overlap – The objects will not stop each others movement, they are treated as non-solid but can still detect and generate overlap collision events.

Split Struct Pin of Hit

SUMMARY

- While there are many ways to access collision data, one of the first you may encounter is through the Add Offset family of nodes.
- Right click on the Sweep Hit Result and either select Split Struct Pin to open up the hit results within the Add Offset Node (pictured Right), or pull off the Sweep Hit Result pin and select Break Hit Result to show the hit results in another node (next page).
- You could also save the hit results in a variable if you wish as the results are simply an Object of data about the collision interaction.



Break Hit Result

BREAK HIT RESULT

- An alternative way to access the hit results. This is the same as the previous slide. This is useful if you want to store or share the hit result however.

The screenshot displays the Unreal Engine Blueprint Editor interface. On the left, a search bar is visible with the text 'Actions taking a(n) Hit Result Structure' and 'Context Sensitive'. Below the search bar, a list of components is shown: Collision, Game, Utilities, and Variables. The 'Break Hit Result' node is highlighted in the search results, with a tooltip that reads: 'Extracts data from a HitResult. Target is Gameplay Statics'.

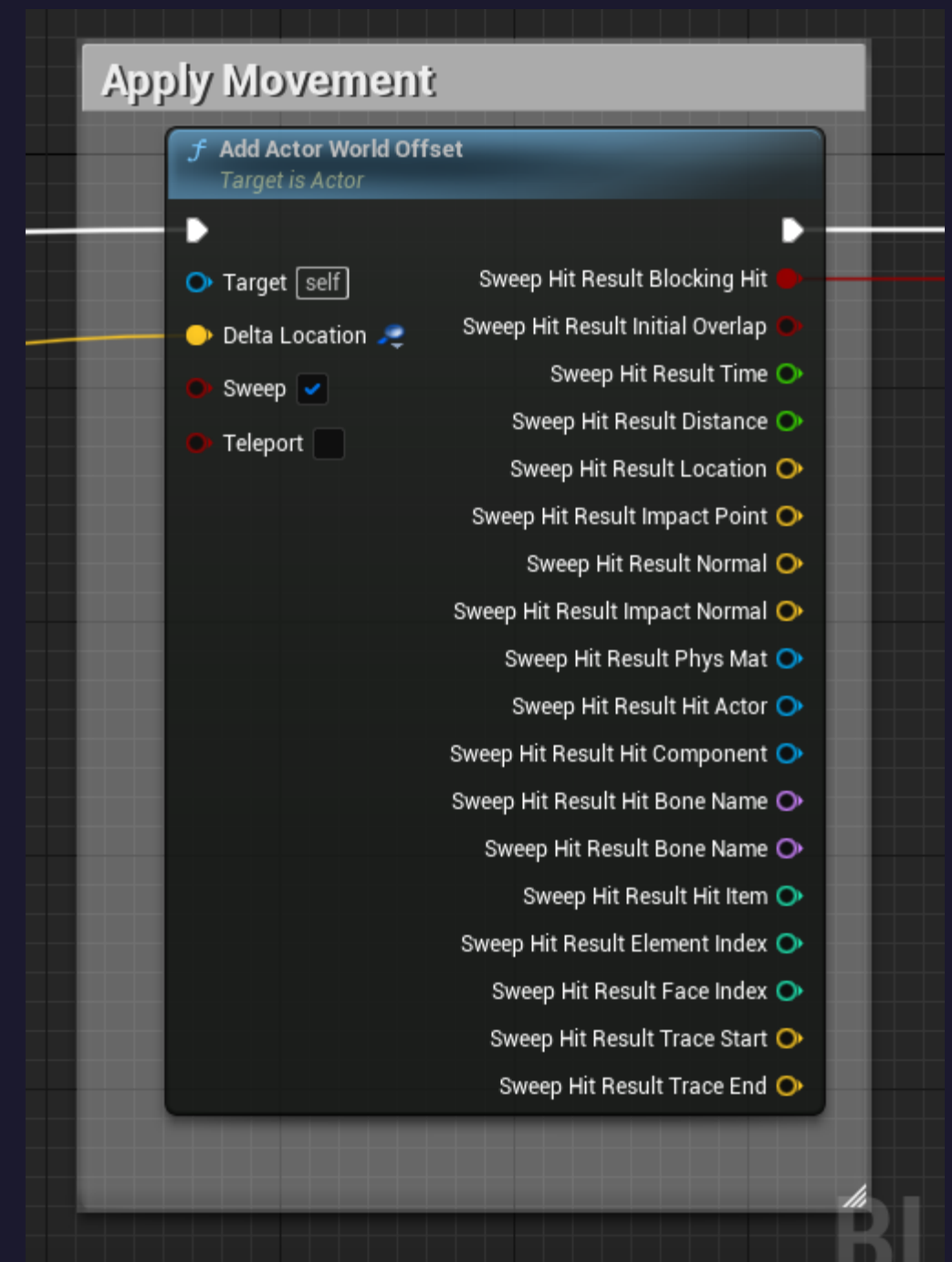
In the center, a 'Break Hit Result' node is shown, which is a blue node with a 'Hit' pin. A blue line connects this node to a 'Sweep Hit Result' node on the right. The 'Sweep Hit Result' node is a blue node with a 'Sweep Hit Result' pin. A blue line connects this node to a 'Break Hit Result' node on the right.

On the right, a 'Break Hit Result' node is shown, which is a blue node with a 'Hit' pin. A blue line connects this node to a 'Sweep Hit Result' node on the right. The 'Sweep Hit Result' node is a blue node with a 'Sweep Hit Result' pin. A blue line connects this node to a 'Break Hit Result' node on the right.

The 'Break Hit Result' node on the right has a list of data pins: Hit, Blocking Hit, Initial Overlap, Time, Distance, Location, Impact Point, Normal, Impact Normal, Phys Mat, Hit Actor, Hit Component, Hit Bone Name, Bone Name, Hit Item, Element Index, Face Index, Trace Start, and Trace End. Each pin has a corresponding icon (a circle with a dot) and a color-coded label.

Useful tools

- Sweep Hit Result Blocking – True if the object moved was Blocked (its movement stopped).
- Sweep Hit Result Initial Overlap – An overlap was started (doesn't stop the movement).
- You can explore the rest of these output pins.
- Some useful ones:
 - Sweep Hit Result Hit Actor – The other Actor we collided with
 - Sweep Hit Result Location / Impact Point – Location we hit, read the tooltip for details
 - Sweep Hit Result Normal/Impact Normal – The Normal (direction) of the collision itself. This can be very useful for aligning particle effects for example, or calculating which direction an object should ricochet off another.



Blocking vs Overlap

- Two main types of collisions we will detect are Blocking and Overlap.
- Blocking Collisions mean the objects will stop each other from intersecting.
- Overlap collisions will NOT stop the objects from intersecting, but will call an Event when they do intersect.
- They are similar, and both rely on the same Collider objects, but there are some subtle differences.

Block vs Overlap

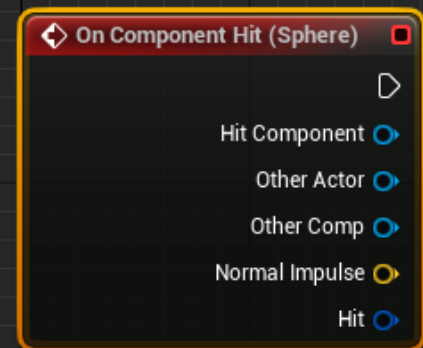
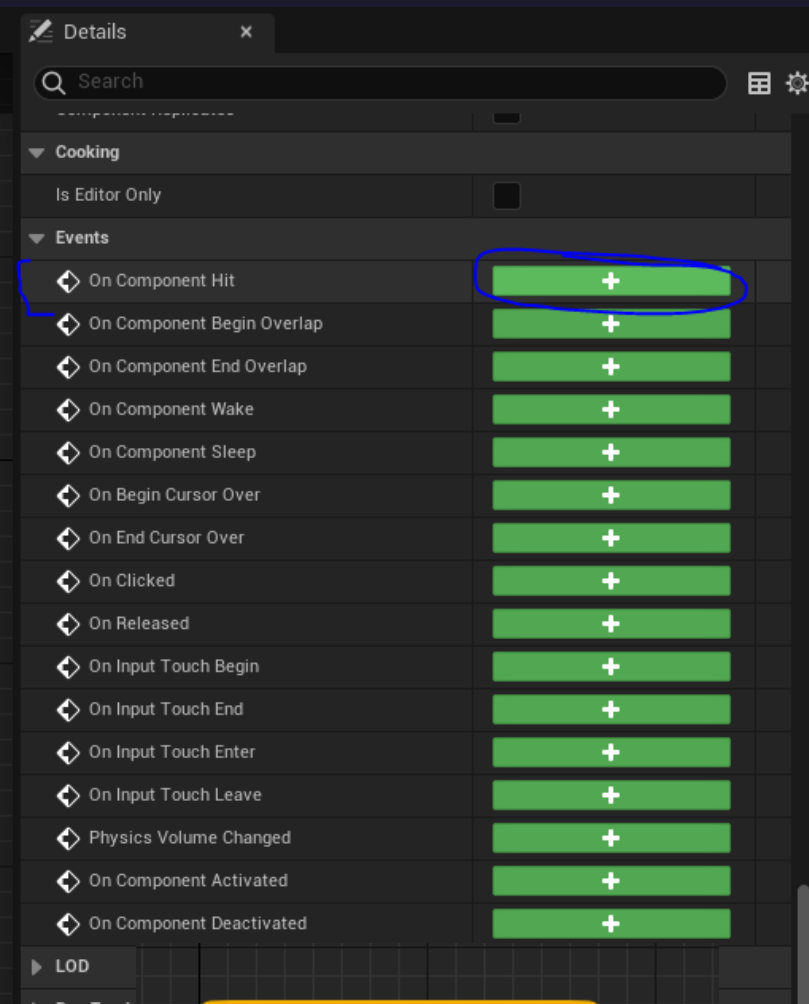
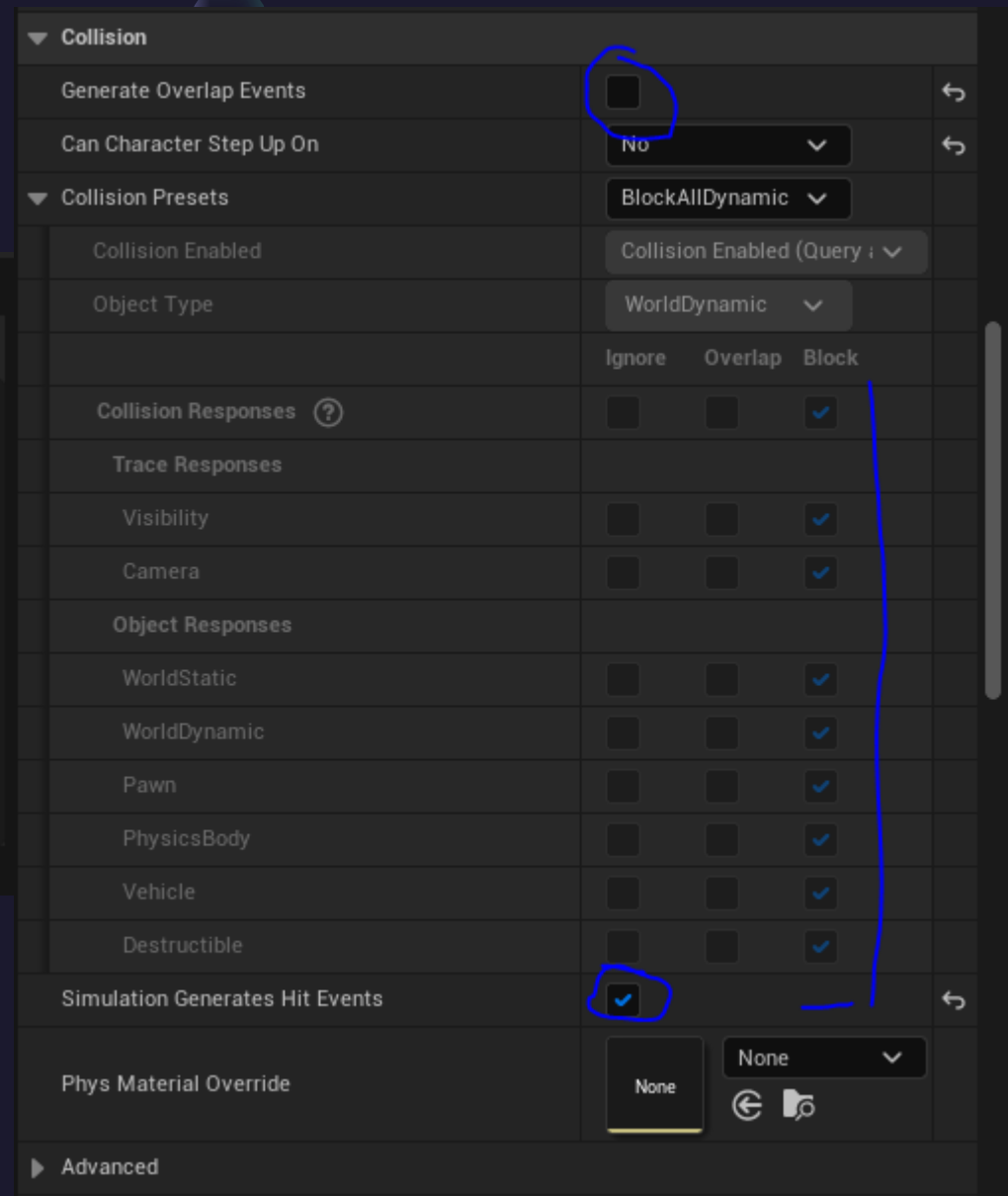
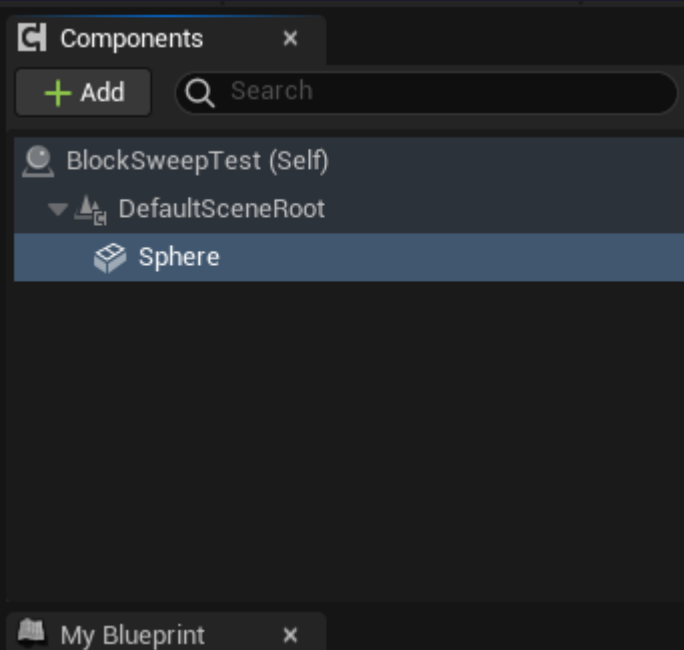
BLOCKING

- Objects will stop movement progress when Sweeping (Add Offset Node) or when using the Physics system.
- Only one object is required to have the `SimulationGeneratesHitEvents` to create a hit event. The object with that = True will create/have the event.
- Blocking will still occur without that checked, it just won't generate an Event for it.
- The Hit Event will be called every physics step that it is colliding. For example, if a Pawn is standing on a floor, that would fire constantly.

OVERLAP

- Objects will NOT stop movement for Overlap collisions.
- BOTH objects must have the Generate Overlap Events checkbox checked for any event to fire OR one object has GOE checkbox = T and the other is set to be a blocking object.
- An Object that is set to Blocking will count as an overlap object for the purposes of generating events.
- Overlap events are ONLY called when the overlap begins and ends, making them much for performant for objects colliding often.

Blocking



- (Clockwise from Top Left)
 - Using a Sphere Component
 - Basic Settings
 - Creating the Hit Event
 - The actual OnComponentHit Event in Blueprints

Overlap

The image displays three screenshots from the Unreal Engine interface, illustrating the setup for sphere overlap events.

Top Left Screenshot: Shows the **Components** panel. The **Sphere** component is selected under the **DefaultSceneRoot** hierarchy.

Top Middle Screenshot: Shows the **Details** panel for the **Sphere** component. The **Collision** section is expanded, showing **Collision Presets** set to **OverlapAllDynamic**. The **Generate Overlap Events** checkbox is checked.

Top Right Screenshot: Shows the **Details** panel for the **Sphere** component. The **Events** section is expanded, showing a list of events. The **On Component Begin Overlap** and **On Component End Overlap** events are highlighted with blue boxes.

Bottom Left Screenshot: Shows the **Event Graph** panel. The **On Component Begin Overlap (Sphere)** and **On Component End Overlap (Sphere)** events are added to the graph. The **On Component End Overlap (Sphere)** event is highlighted with a blue box.

- (Clockwise from Top Left)
 - Using a Sphere Component
 - Basic Settings
 - Creating the OnComponentBegin/End Overlap events
 - The actual Overlap Events in Blueprints

QUOTED FROM THE DOCS:

[HTTPS://DOCS.UNREALENGINE.COM/4.26/EN-US/INTERACTIVEXPERIENCES/PHYSICS/COLLISION/OVERVIEW/](https://docs.unrealengine.com/4.26/en-us/interactivexperiences/physics/collision/overview/)

- There are a few rules to keep in mind with how collisions are handled:
- Blocking will naturally occur between two (or more) Actors set to Block. However, Simulation Generates Hit Events needs to be enabled to execute Event Hit, which is used in Blueprints, Destructible Actors, Triggers, etc...
- Setting Actors to Overlap will often look like they Ignore each other, and without Generate Overlap Events, they are essentially the same.
- For two or more simulating objects to block each other, they both need to be set to block their respective object types.
- For two or more simulating objects: if one is set to overlap an object, and the second object is set to block the other, the overlap will occur but not the block.
- Overlap events can be generated even if an object Blocks another, especially if traveling at high speeds.
 - It is not recommended for an object to have both collision and overlap events. Though possible, there is much that needs manual handling.
- If one object is set to ignore and the other is set to overlap, no overlap events will be fired.

Test Yourself

- In Class:
- Reproduce each of the 4 scenarios listed on the Collision Overview page of the doc
 - Collision
 - Collision with Simulation Generates Hit Events
 - Overlap and Ignore
 - Overlap and Generate Overlap Events
- <https://docs.unrealengine.com/4.26/en-US/InteractiveExperiences/Physics/Collision/Overview/>

Test Yourself C++ Edition

- You can also handle Collision Events in C++ code by providing the appropriate Function.
- <https://unrealcpp.com/on-component-hit/>

Traces and Queries

- We have mostly looked at collisions in the context of movement, either by the Physics system or using Add Offset, however you can also use a variety of queries to make a specific collision test on a specific frame.
- Broadly speaking there are 3 main categories:
 - Line Trace: Also called a Raycast, this is like shooting a laser instantaneously.
 - Sphere/Box/Capsule Overlap: This tests a specific shape at a specific location for overlaps.
 - Sphere/Box/Capsule Sweep: This is like a combination of a Raycast and a Overlap. It moves a shape along a path and detects any collisions along the way. These can be particularly useful when trying to plan a path for a character to move or for detecting larger collision areas.

Traces and Queries

- There are several variations of each but broadly speaking there are a few key points.
- A line/sphere Trace will stop the trace and return a reference to the first object hit when it collides with something.
- You can use a Multi Trace to return all objects intersected with. It is a little more complex as you will have to retrieve the specific object you want yourself.
- There are two broad categories of filtering, for Objects or by Channel.
 - By Objects is for detecting scene objects or if you have created a custom Object Channel.
 - By Channel is for things that are queried but not blocking, or when you have created a custom Trace Channel.
- <https://docs.unrealengine.com/4.26/en-US/InteractiveExperiences/Tracing/>

Custom Channels

▼

Engine - Collision

Set up and modify collision settings.

Export...

Import...

🔒

These settings are saved in DefaultEngine.ini, which is currently writable.

▼

Object Channels

You can have up to 18 custom channels including object and trace channels. This is the list of object types for your project. If you delete an object type that is being used by the game, any uses of that type will revert to WorldStatic.

New Object Channel...

Edit...

Delete...

Name

Default Response

CustomOverlapObjectChannel

Overlap

▼

Trace Channels

You can have up to 18 custom channels including object and trace channels. This is the list of trace channels for your project. If you delete a trace channel that is being used by the game, the behavior of the trace is undefined.

New Trace Channel...

Edit...

Delete...

Name

Default Response

CustomOverlapTraceChannel

Overlap

▼

Preset

You can modify any of your project profiles. Please note that if you modify profile, it can change collision behavior. Please be careful when you change currently existing (used) collision profiles.

New...

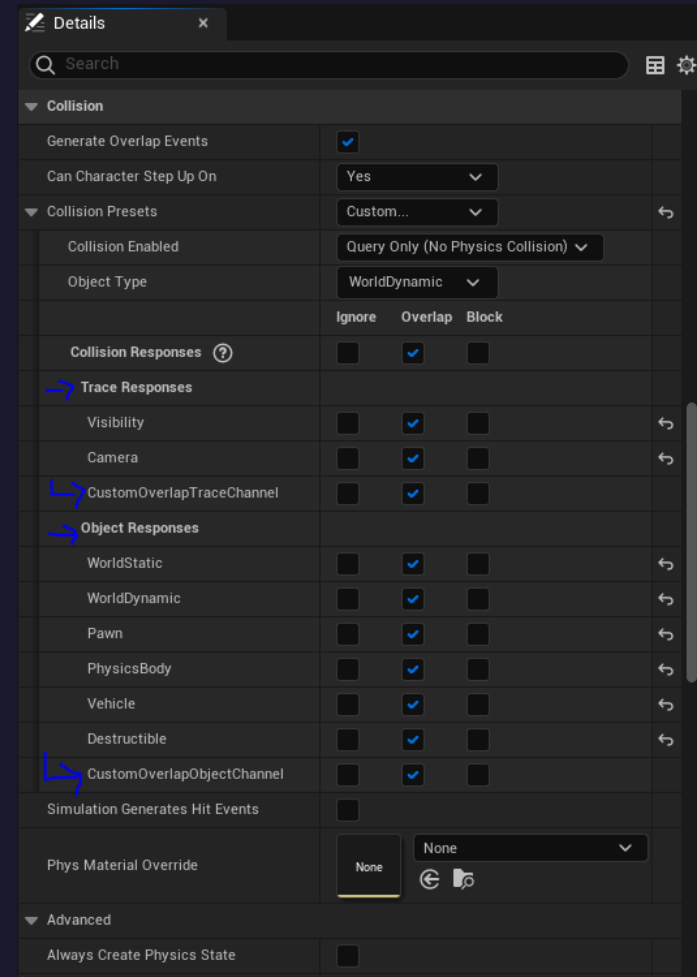
Edit...

Delete...

	Name	Collision	Object Type	Description
⚙️	NoCollision	No Collision	WorldStatic	No collision
⚙️	BlockAll	Collision Enabled (Query and PhysWorldStatic		WorldStatic object that blocks all actors by default. All new custom c
⚙️	OverlapAll	Query Only (No Physics Collision)	WorldStatic	WorldStatic object that overlaps all actors by default. All new custom
⚙️	BlockAllDynamic	Collision Enabled (Query and PhysWorldDynamic		WorldDynamic object that blocks all actors by default. All new custoi
⚙️	OverlapAllDynamic	Query Only (No Physics Collision)	WorldDynamic	WorldDynamic object that overlaps all actors by default. All new cust
⚙️	IgnoreOnlyPawn	Query Only (No Physics Collision)	WorldDynamic	WorldDynamic object that ignores Pawn and Vehicle. All other chann
⚙️	OverlapOnlyPawn	Query Only (No Physics Collision)	WorldDynamic	WorldDynamic object that overlaps Pawn, Camera, and Vehicle. All ot

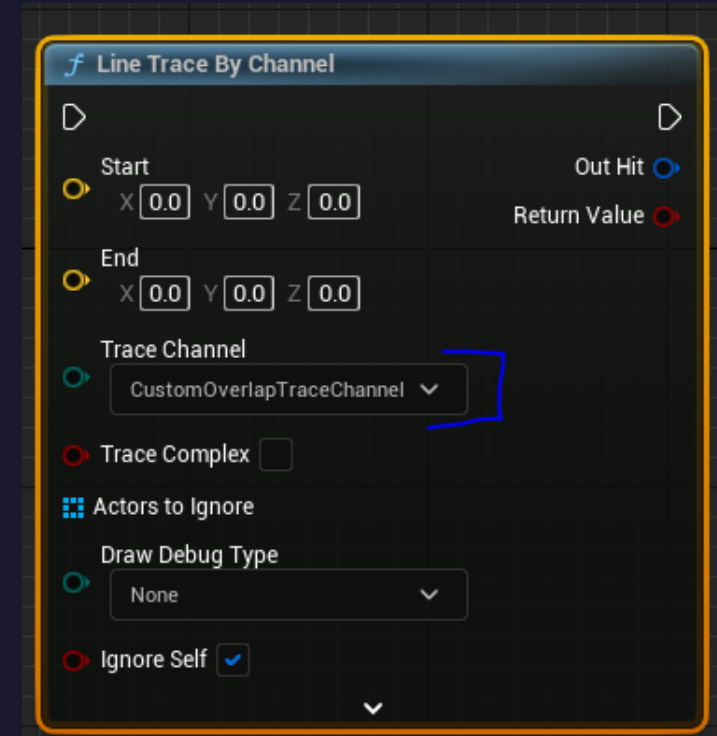
Traces and Queries

- Custom Object and Trace channels are set up in Project Settings > Collision.
- As you can see, both Custom Object and Trace channels appear in the Collision Details panel.
- Trace channels are for non-scene objects (visibility, Camera, etc) while Object Channels are for scene objects.



Traces and Queries

- Using Line Trace by Channel will allow you to select a given Channel to trace.



Traces and Queries

- Using Line Trace by Object will allow you to select from a list of Object types to filter.

The screenshot displays the Unreal Engine Blueprint Editor interface. The central viewport shows a grid with several nodes: 'Line Trace For Objects', 'Line Trace by Object', 'Sphere Overlap Actors', and 'Box Overlap Actors'. The 'Line Trace by Object' node is selected, and its configuration is visible in the 'Details' panel on the right. The 'Object Types' variable is set to 'EObject Type'. The 'Default Value' section shows a list of object types: 'Pawn', 'Vehicle', and 'CustomOverlapObjectChannel'. A blue arrow points to the 'CustomOverlapObjectChannel' entry. The left sidebar shows the 'Components' panel with 'Object Types' selected under 'Variables'. A blue line connects the 'Object Types' variable in the 'Line Trace by Object' node to the 'Object Types' variable in the 'Details' panel.

Line Trace by Object Node Configuration:

- Start: X [0.0] Y [0.0] Z [0.0]
- End: X [0.0] Y [0.0] Z [0.0]
- Object Types: [Grid Icon]
- Trace Complex: []
- Actors to Ignore: [Grid Icon]
- Draw Debug Type: [None]
- Ignore Self: [x]

Details Panel - Variable:

Variable Name	Object Types
Variable Type	EObject Type
Description	
Instance Editable	[]
Blueprint Read Only	[]
Expose on Spawn	[]
Private	[]
Expose to Cinematics	[]
Category	Default
Replication	None
Replication Condition	None

Details Panel - Advanced - Default Value:

Object Types	3 Array elements
Index [0]	Pawn
Index [1]	Vehicle
Index [2]	CustomOverlapObjectChannel

Further Reading

- <https://www.unrealengine.com/en-US/blog/collision-filtering>
- <https://zompidev.blogspot.com/2021/08/visibility-vs-camera-trace-channels-in.html>
- <https://docs.unrealengine.com/4.27/en-US/InteractiveExperiences/Physics/Collision/HowTo/PreviewCollision/>